Reading Before Writing:

AI Discipline for Legacy Code: *A hypothesis for testing*

Abstract

This paper proposes that the high-leverage application of AI to legacy software is not editing but documentation. It argues that the bottleneck on safe AI-assisted change in mature systems is not code-generation capability but the surfacing of tacit knowledge held by experienced engineers and not represented in the codebase. It proposes an interrogative role architecture, operating under strict non-editing discipline, and offers this as a falsifiable hypothesis to be tested against a real legacy codebase over a defined period. Where the previous papers in this sequence established a mapping framework and tested it against a greenfield project, this paper asks whether the same framework applies to the harder case, where most production software actually lives.

1. The greenfield bias

Most reports on AI-assisted software development describe greenfield work. A new project, a fresh repository, a defined scope, a single author or a small team with shared conventions — these are the conditions under which AI coding tools demonstrate their most impressive capabilities. The output is real, the productivity gains are real, the methodology is teachable.

The conditions are also not representative. Most software engineering, by volume of person-hours, by impact on revenue, by exposure to risk, takes place

inside legacy codebases. Years or decades of accumulated decisions, misunderstandings, deferred fixes, vendor constraints, organisational reorganisations, half-completed migrations, and tacit knowledge held by individuals who may or may not still be in the building. The code does not represent its own intent; it represents a long history of intent, partially expressed and partially eroded.

If AI assistance is genuinely useful only on greenfield, the field has a problem. Greenfield is the minority case. The question that matters for the broader claim about AI productivity in software is not whether a small team can build a new system efficiently with AI assistance. The question is whether AI assistance can be applied to existing systems — the ones that hold the data, run the business, and account for the bulk of engineering effort — without making them worse. The economic logic is unforgiving: an AI capability that works only on the small minority of work where most projects do not live is a curiosity, not a productivity tool.

2. What makes legacy different

A legacy codebase differs from a greenfield project in several ways that matter structurally rather than incidentally.

The patterns visible in the code may not be patterns one wants to reproduce. Most codebases of any age contain stylistic compromises, expedient solutions, half-applied refactors, and outright bugs that have become load-bearing. An AI assistant that defaults to matching the style of surrounding code — which is

the natural and usually correct behaviour on greenfield — will, on legacy, propagate exactly those patterns one is trying to escape. The gradient of "match what is here" pulls toward the worse code, not the better.

The cost of change is high and the tolerance for error is low. Many legacy systems have integration constraints that make casual modification dangerous in non-obvious ways. A formatting change that would be innocuous in a fresh project can, in a system whose deployment pipeline generates idempotency hashes from file contents, cause deployment to fail. A "tidy-up" of a configuration block that looks redundant can trigger a migration affecting hundreds of thousands of records. The surface appearance of a change tells the engineer little about its actual consequence, and the AI assistant operating on the surface alone has no way to recover the difference.

The knowledge required to operate safely is largely tacit. It lives in senior engineers' heads, in the oral tradition of the team, in incident postmortems that may or may not have been written down. The codebase itself documents what was done, not why; rarely what must not be done. The constraints are learned by experience, transmitted by apprenticeship, enforced by social correction. None of this is legible to an AI assistant reading the code in isolation.

The result, anecdotally and increasingly visibly, is that engineers attempting to apply AI assistance to legacy work report consistent failure modes: drift away from local conventions toward generic patterns; misinterpretation of bad

patterns as good, on the implicit reasoning that a pattern repeated must be a pattern intended; inadequacy of existing review and tooling mechanisms to enforce constraints the AI cannot see; and a kind of fluent confidence in output that disguises the absence of grounding. The problem is not that AI is incapable of writing code. The problem is that the code it writes is plausible without being correct, and on legacy the gap between plausible and correct is wider and more consequential than on greenfield. These failure modes are common enough among working engineers to be folklore. Folklore is not a basis for understanding what works.

3. The mapping argument, applied

A previous paper in this sequence argued that AI agents do not reduce the need for conceptual clarity; they expose its absence and scale its consequences. Roles understood through convention, apprenticeship, and social feedback do not survive externalisation to an agent that operates over explicit representations only. The corrective is to model human activities explicitly before mapping them, decomposing roles into informational transformations (which an agent can perform) and irreducibly human functions (which it cannot).

The application to legacy code is direct, and arguably the original framework's most consequential implication. A legacy codebase is a system whose conceptual clarity is partial at best. Its ubiquitous language is incomplete; its invariants are partly enforced and partly hoped for; its constraints are partly documented and partly traditional; its intent is partly expressed in code and partly remembered. An AI assistant introduced to such a codebase encounters

precisely the conditions the mapping framework predicts will produce locally plausible, globally misaligned output. The plausibility is the danger. The absence of conceptual clarity is the cause. The assistant's fluency is what makes it look as if everything is fine when it is not.

The implication is sharp. Before AI assistance can safely be applied to editing legacy code, the conceptual clarity it requires must be established. The clarity does not exist in the code, and it cannot be invented by the assistant. It must be surfaced from the people who hold it, made explicit, structured, cross-referenced, and made available as context to subsequent work. This is documentation work, and it is what the high-leverage application of AI to legacy systems looks like.

4. Reading before writing

The discipline this paper proposes is interrogative, not generative. The AI assistant is used to read, structure, and question; not to write code. It operates in four activity modes, each in its own session, each with defined output formats.

In cold reading, the assistant analyses pieces of code without human presence, producing reports that document apparent purpose, named concepts, observable behaviour, dependencies, suspicious patterns, and (most importantly) explicit lists of questions that cannot be derived from the code itself. The questions are the load-bearing output. They are the surface area of tacit knowledge. They are what an interview can address.

In interview support, the assistant participates in structured questioning sessions in which a senior engineer answers the open questions surfaced by cold reading, with the transcript captured faithfully and contradictions or gaps flagged. The questioning style favours open over closed, examples over principles, consequence questions ("if I changed this, what would break?") over description questions ("what does this do?"). The interviewer captures; it does not editorialise.

In synthesis, the assistant combines cold readings and interviews into candidate documents in defined formats: constraints, hazards, glossary entries, retrospective architectural decision records. The candidates are reviewable; they are not yet canonical.

A human curator (operated as a discipline, not as an agent role) decides which candidates are promoted to canonical, flags contradictions across the corpus, and maintains an index. The authority to canonicalise is reserved for humans, on the principle that judgement about correctness is one of the human functions the mapping framework explicitly does not delegate.

A separate role, the test planner, applies the same cold-reading discipline to support a test team in reverse-engineering specifications from existing behaviour, producing structured test plans with explicit flags about whether scenarios characterise observed behaviour or validate documented requirements. The distinction matters: on a legacy codebase with no documented requirements, almost every test will be of the first kind, and

writing them as if they were of the second produces exactly the false confidence that makes legacy systems harder to change rather than easier.

None of these roles edits code. The discipline is strict on this point, partly for the obvious safety reason — uncontrolled AI editing of legacy code is the failure mode this whole architecture exists to prevent — and partly for a deeper reason: separating the activity of understanding from the activity of changing keeps the understanding honest. An interrogator that can also edit will be tempted, in subtle ways, to read the code in the light of the changes it might make. An interrogator that cannot edit reads the code as it is.

5. The corpus as artefact

The output of this discipline, if it works, is a corpus of structured, cross-referenced documents that did not previously exist. Constraints and hazards explicitly named. Glossary entries that distinguish local meanings from vendor meanings, or one team's usage from another's. Retrospective architectural decision records that reconstruct the contexts of decisions made years ago and document the consequences that followed. An index that makes the corpus navigable.

The corpus has value independent of AI. A team that can point to a hazard register listing the operations that look benign but are not, with mitigation patterns described, has reduced the operational risk of working with the codebase regardless of whether anyone uses AI on it again. A glossary that distinguishes the vendor's "balance" from the local "balance" prevents

confusion in design conversations whether or not the next change is AI-assisted. The corpus is a documentation artefact whose justification stands without reference to AI tooling.

The corpus also constrains future AI use. Once it exists, an AI assistant operating in any role can be required to read the relevant corpus entries before acting. Constraints become available to context engineering. Hazards become known. The failure modes that follow from AI not knowing what experienced engineers know become tractable, because the knowledge is no longer trapped.

This is the testable claim that organises the experiment: the corpus precedes the editing. AI editing on legacy code that lacks a sufficient corpus is an importation of all the failure modes the previous paper described, scaled by the legacy codebase's own pre-existing absence of conceptual clarity. AI editing on legacy code where a sufficient corpus exists is a different proposition. It remains subject to the discipline the mapping framework requires, but with the necessary context now available.

6. Falsifiable claims

A hypothesis is worth holding only if it can be falsified. This paper makes five claims, each phrased to allow the experiment that follows to refute it.

First, that on legacy systems the high-leverage application of AI is documentation rather than editing. This is falsified if a month of interrogative

work produces a thin or low-utility corpus, or if parallel observation suggests that editing-focused AI use produces equivalent value at lower overhead.

Second, that the interrogative role architecture (cold reading, interview, synthesis, curation as separated modes; strict non-editing discipline; structured output formats) produces qualitatively different outputs from free-form AI engagement on the same material. This is falsified if a controlled comparison shows no detectable structural advantage.

Third, that the conceptual clarity requirement of the mapping framework applies with particular force to legacy code, such that the corpus produced is as much a documentation of the team's prior understanding as it is a documentation of the codebase. This is falsified if the corpus turns out to be straightforwardly extractable from the code itself, with little tacit content surfaced.

Fourth, that the bottleneck on safe AI-assisted change in mature systems is the tacit knowledge gap, not the code-generation capability. This is falsified if limited editing work performed during the experiment produces equivalent quality to what would be expected post-corpus, or if the corpus fails to constrain editing in any observable way.

Fifth, that the interrogator architecture is consistent with the mapping framework: it decomposes the human activity of understanding a legacy codebase into informational transformations mappable to AI, while preserving the irreducibly human functions outside the agent. This is falsified if the role

decomposition fails operationally — modes blurring under pressure, judgement leaking into agent output, authority decisions being silently made by the corpus rather than by humans.

7. What this paper does not claim

The proposed discipline is not a replacement for engineering judgement. It is an architecture within which engineering judgement is exercised more efficiently, with better-grounded context and clearer boundaries. The corpus does not decide; humans decide, with the corpus as reference.

The architecture is not assumed to scale to teams without the discipline that produced it. The previous paper in this sequence was honest about this limit on greenfield work, and the same caveat applies here with greater force. The experiment proposed is a small, senior-engineer-led exercise; not a team-wide rollout. Whether the same framework can survive in less disciplined hands is a further question, and not one the experiment can answer.

The proposed discipline is not, finally, a defence of the legacy codebase as it is. The corpus produced may make perfectly clear that the codebase is structurally unsound, that certain components require replacement, that load-bearing accidents have accumulated to the point of operational risk. These are findings, not failures of the discipline. The discipline's job is to surface what is true, not to flatter what is.

8. Why this matters

The broad claim about AI productivity in software depends on AI being usable across the range of software engineering, not only on the greenfield minority. The case studies that have shaped industry expectations — fresh projects, well-defined scopes, small disciplined teams — are not representative of where the engineering hours and the business value actually sit.

If the interrogative discipline (or something structurally similar) turns out to be necessary for safe AI use on legacy systems, the implications are practical and immediate: organisations introducing AI to legacy work without it should expect the failure modes the mapping framework predicts, scaled by the absence of corpus. The folklore of drift, pattern misinterpretation, and constraint violation will remain folklore — common, costly, and unaddressable, because the architecture that would address it is not in place.

If the discipline turns out to be unnecessary, in the sense that AI editing tools become capable enough that legacy codebases can be edited safely without prior tacit-knowledge surfacing, the experiment will produce evidence of that too. Either result is informative. Either result advances what is currently a stalemate between enthusiastic adoption and resigned dismissal, neither of which is grounded in disciplined observation.

The experiment proposed here will run for approximately one month, on a real legacy codebase, with three senior engineers and a test team. The output will be a corpus, a set of metrics, a surprises log, before-state belief documents, and

— eventually — a paper reporting what happened against what this paper predicted.

The current paper is the prediction. The next will be the result.